



北京大学

《量子物理学导论》课程论文

题目：从? 开始的量子程序语言设计

学号 姓名：2400012921 于孟宏

二〇二六年 八 月

目 录

第一章	如何编写/验证一个量子程序?	1
1.1	量子编程语言的语法	2
1.2	量子编程语言的操作语义	3
1.3	量子霍尔逻辑	4
1.4	总结和展望	5
第二章	量子物理学导论: 这门课教会了我什么	6
第三章	附录 A: 我目前对 trace 的理解	7
参考文献		8

第一章 如何编写/验证一个量子程序?

众所周知, 信息科学技术学院划分为电子工程 (EE) 与计算机科学 (CS) 两个方向, 这源于现代计算体系在硬件 (电路) 与软件 (程序) 层面的分工: 硬件提供物理计算能力, 而软件则在此基础上进行逻辑表达与运算. 如今, 归功于现代编程语言所提供的语法与语义抽象, 软件工程师已无需掌握底层电路实现的全部细节. 例如, 开发者可以在 Python、C/C++、Haskell 或 Rust 等高层语言中直接编写表达式 $a + b$, 而无需关心底层加法器电路的物理实现.

随着量子计算逐步成为未来算力演进的重要方向, 在量子计算模型之上建立抽象的编程语言与编译体系显得意义非凡. 一方面, 这可以将开发者从复杂的底层物理机制中解放出来 (也许未来的量子计算程序员无需深入薛定谔方程的细节, 仅需知晓他们可以在量子比特上施加酉变换即可); 另一方面, 这也为在量子计算模型上进行形式化验证提供了基础. 所谓形式化验证, 意在以逻辑证明等手段保证一段程序的绝对正确性, 现已在编译器等对正确性有较高需求的软件中起到了重要地位, 典型工作如 CompCert^[1]. 而量子程序比起普通程序, 往往更加困难和不直观, 因此可能会有更多人类难以察觉的微小错误, 这就为形式化验证提供了发展空间.

已经有相当多的程序语言领域学者对此进行过深入研究. 在程序语言设计方面, 已经有诸如命令式语言 QCL^[2]和 qGCL^[3]以及函数式语言 QPL^[4]和 QML^[5]等成果; 而在形式化验证领域, 相当重要的工作当属应明生老师将霍尔逻辑推广到量子计算上的论文^[6]. 这篇论文不仅给出了一套量子计算的形式化语法和语义, 还在上面建立了量子霍尔逻辑的理论用于形式化证明 (然而有整整 49 页正文还全是公式我真的读自闭了). 本篇报告的后续部分都将基于此论文, 可以看作是本人读完后对该论文的一点微小 (还可能出错) 的理解, 许多图和公式直接从原论文中摘取, 恕不逐个引用. 此外, 也有不少公司在此方面实现了一些量子模拟语言, 例如微软的 Q#^[7], Google 的 Cirq^[8]以及 IBM 的 Qiskit^[9].

我们将在 [1.1] 一节中简介量子编程语言的一套基本语法, 并在 [1.2] 中简介其操作语义; 最后我们会在 [1.3] 中介绍如何使用量子霍尔逻辑进行形式化验证. 不过在此之前, 我们将先用一个简单程序介绍这三个概念在常规编程语言中的应用.

我们来看一个很简单的语句: $a = a + 1$:

- 所谓**语法**, 是说限制你可以写出什么东西. 例如你可以在 C++ 语言中写 $++a$ 表示 $a = a + 1$, 但在 C 语言中不能. 再比如, C 语言的语法告诉我们可以 C 语言中写出赋值语句 $a = 711$, 分支语句 $\text{if}(a == 1)a = 2$; 循环语句 $\text{while}(a ==$

0) $a = a - 1$; 等等. 常用 BNF 范式表达一个语言的语法 (一个 BNF 范式表达的是一个表达式可以有哪些可能, 比如 $A ::= B|C|D$, 意味着 A 语句可以取 B, C, D 中的任意一种).

- 所谓**操作语义**, 可以理解为需要定义一个程序的”状态”, 在经过一个语句后会发生怎么样的变化. 例如我们可以写: $\{a = 0\}a = a + 1; \{a = 1\}$ 来表示: 变量 a 一开始是 0, 在经过了 $a = a + 1$; 语句操作后, 变量 a 成为了 1. 这就提供了对语句 $a = a + 1$ 的一个操作语义. 常用自然演绎范式来表示. 这种范式的特征是有一条长长的横线, 横线上面是前提假设, 横线下面是我们需要的结论 (例如 $\frac{A \rightarrow B \quad A}{B}$, 表示的是当我们有前提假设 $A \rightarrow B$ 和 A 时, 我们就可以得到结论 B).
- 所谓**霍尔三元组**, 可以理解为现在我们可以用逻辑谓词来描述当前的程序状态, 比如 $\{0 \leq a \leq 2\}a = a + 1; \{1 \leq a \leq 3\}$. 表示的是如果我们知道在执行 $a = a + 1$; 之前, 变量 a 满足 $\{0 \leq a \leq 2\}$. 那么在执行 $a = a + 1$; 之后, 变量 a 满足 $\{1 \leq a \leq 3\}$.

1.1 量子编程语言的语法

要想设计一门编程语言, 首先就要设计其语法. 例如, 一个最简单的命令式语言的语法可以写成:

(算术表达式 Arithmetic Expr) $e ::= n \mid x \mid e_1 + e_2 \mid e_1 \times e_2$

(布尔表达式 Boolean Expr) $b ::= \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 \leq e_2 \mid \neg b \mid b_1 \wedge b_2$

(程序语句 Statements) $S ::= \text{skip} \mid x := e \mid S_1; S_2 \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \mid \text{while } b \text{ do } S$

而对于一门量子编程语言, 最大的不同是我们无法在不测量的时候判断一个 qubit 处于什么状态, 因此, 控制流 (if/else, while) 必须写成测量的形式, 如下:

$S ::= \text{skip} \mid q := 0 \mid \bar{q} := U\bar{q} \mid S_1; S_2 \mid \text{measure } M[\bar{q}] : \bar{S} \mid \text{while } M[\bar{q}] = 1 \text{ do } S$

图 1.1 Syntax

其中, **measure** 语句中的 $M = \{M_m\}$ 类似 C++ 语言中的 switch 语句, 它将会使用一组满足 $\sum_m M_m^\dagger M_m = I$ 的测量矩阵来判断当前的程序状态并送入不同的分支 $\bar{S} = \{S_m\}$. 而 while 语句中的 $M = \{M_0, M_1\}$ 则用来指定循环条件. 容易见到, 这里程序会以不同的概率进入不同的分支, 并继续接下来的运算. 因此我们必须刻画程序进入不同分支的概率.

1.2 量子编程语言的操作语义

使用密度矩阵 (density operator) 描述当前程序的状态看上去是一个好主意. 相比于用叠加态向量 $\psi = \sum_i \psi_i$ 来表示程序状态, 密度矩阵可以更好地刻画混合态. 这是由于密度矩阵总是 Hermitian 矩阵而且满足 $\text{trace} = 1$, 使其总可以拆成 $\sum_i p_i |\psi_i\rangle\langle\psi_i|$ 的形式, 可以理解为当前状态以 p_i 的经典概率为纯态 ψ_i . 然而有趣的是, 由于我们想要刻画一个程序走到不同分支的概率, 以及我们甚至可能还希望刻画一个程序是否总能停机 (也许它会陷入死循环?), 在这里我们使用偏密度矩阵 (partial density operator) 来表示程序状态, 用 $\mathcal{D}^-(\mathcal{H})$ 表示它们的集合, 它的唯一不同点在于它允许 $\text{trace} \leq 1$. 如果用于表示当前的程序状态的偏密度矩阵 $\text{trace} = p$, 意味着当前程序有 p 的概率执行到这一步. 这就方便我们刻画一个 while 循环是否总会结束 (比如有多少的概率会成为死循环) 或者一个 if 分支语句进入每个分支的概率.

我们知道可以用 Hermitian 线性算符去测量一个物理量, 对于一组测量矩阵满足 $\sum_m M_m^\dagger M_m = I$, 我们可以证明当前程序状态 (用偏密度矩阵 ρ 表示), 我们可以证明测量 ρ 落在状态 $M_m^\dagger M_m$ 的概率恰为 $\text{trace}(M_m^\dagger M_m \rho)$. 而测量完之后, 当前的状态必须坍缩到 M_m 所代表的可能性状态之一, 因此将会坍缩到 $\rho' = \frac{M_m \rho M_m^\dagger}{\text{trace}(M_m^\dagger M_m \rho)}$.

不得不在这里插一句嘴是, 我在上述的这些定义中徘徊了三四天, 遇到的最主要的问题就是我不理解这里为什么会需要 trace. 现在我认为我对这个问题有了一个较为满意的答案, 可以参见附录 [三].

现在我们拥有了表示程序的工具 (语法), 以及表示状态的工具 (偏密度矩阵). 我们将使用 $\langle S, \rho \rangle$ 符号表示: 当前程序状态是 ρ , 即将执行程序 S . 我们用 E 表示空程序, 并用 $\langle S, \rho_1 \rangle \rightarrow \langle E, \rho_2 \rangle$ 表示: 程序状态 ρ_1 在执行完 S 之后, 来到了程序状态 ρ_2 . 出于篇幅考虑, 我们不会在这里介绍所有的操作语义, 而是只介绍几种作为例子:

$$(Skip) \quad \overline{\langle \text{skip}, \rho \rangle \rightarrow \langle E, \rho \rangle}$$

图 1.2 Skip

Skip 的操作语义非常简单: 经过 skip 语句后, 当前程序状态保持不变.

$$(Unitary \ Transformation) \quad \overline{\langle \bar{q} := U\bar{q}, \rho \rangle \rightarrow \langle E, U\rho U^\dagger \rangle}$$

图 1.3 Unitary Transformation

Unitary 操作, 实际上也就是薛定谔方程中的时间演化算符, 它会把状态从 ρ 转化为 $U\rho U^\dagger$.

$$(Measurement) \quad \frac{}{\langle \mathbf{measure} \ M[\bar{q}] : \bar{S}, \rho \rangle \rightarrow \langle S_m, M_m \rho M_m^\dagger \rangle}$$

for each outcome m of measurement $M = \{M_m\}$

图 1.4 Measurement

Measurement 比较特殊: 如果一个程序状态通过了 M_m 的测量, 那么首先它将坍缩到 $\rho' = \frac{M_m \rho M_m^\dagger}{\text{trace}(M_m^\dagger M_m \rho)}$. 然而回忆到我们引入偏密度矩阵的动机: 我们要用它的 trace 来刻画程序到达当前分支的概率, 因此对于每个分支, 偏密度矩阵都会变为 $M_m \rho M_m^\dagger$.

1.3 量子霍尔逻辑

最后我们要引入量子霍尔逻辑. 通常的霍尔逻辑中需要谓词的概念, 例如 $\{0 \leq a \leq 2\}a = a + 1; \{1 \leq a \leq 3\}$ 中, $0 \leq a \leq 2$ 就是一个谓词. 霍尔逻辑推导规则就是在告诉我们一个刻画当前程序状态的谓词, 在经过一个语句后, 会变成怎样的谓词. 我们可以在量子程序上定义类似的谓词, 我们要求一个谓词是 Hermitian 矩阵, 并且满足对于任意 $\rho \in \mathcal{D}^-(\mathcal{H})$, 总有 $0 \leq \text{tr}(A\rho) \leq \text{tr}(\rho)$. 也就是说: 这个谓词成立的概率, 必须 ≥ 0 , 同时 $\leq$ 程序成功运行到这里的概率. 借此, 我们得以建立我们的量子霍尔三元组. 当我们写 $\{P\}S\{Q\}$ 时, 我们要求:

- 对于任意偏密度矩阵 ρ_1 , 如果其在经过程序 S 的操作后, 会变成偏密度矩阵 ρ_2 .
- 一定有 $\text{tr}(P\rho_1) \leq \text{tr}(Q\rho_2)$

直观地理解的话: 对于一个一开始以 $\text{tr}(P\rho_1)$ 概率满足 P 的状态 ρ_1 , 它在经历过程序 S 的操作后变成了 ρ_2 , 此时它满足 Q 的概率至少不会降低.

我们可以来看一看在这种情况下, 我们该怎么对程序进行推导. 我们将介绍几种推导规则作为例子:

$$(Axiom \ Skip) \quad \{P\} \mathbf{Skip} \{P\}$$

图 1.5 Axiom Skip

Skip 的霍尔逻辑同样十分简单: 如果一开始这个程序以一定的概率满足 P , 则由于 skip 什么都不变, 后续还会以相同的概率满足 P .

$$(Rule\ Sequential\ Composition) \quad \frac{\{P\}S_1\{Q\} \quad \{Q\}S_2\{R\}}{\{P\}S_1; S_2\{R\}}$$

图 1.6 Rule Sequential Composition

这也很好理解: 假设 $\rho_1 \xrightarrow{S_1} \rho_2 \xrightarrow{S_2} \rho_3$, 横线上方的前提意味着 $tr(P\rho_1) \leq tr(Q\rho_2) \leq tr(R\rho_3)$, 因此自然也就导出了 $tr(P\rho_1) \leq tr(R\rho_3)$.

$$(Axiom\ Unitary\ Transformation) \quad \{U^\dagger P U\} \bar{q} := U \bar{q} \{P\}$$

图 1.7 Axiom Unitary Transformation

假设一开始的程序状态为 ρ , 我们知道此时满足 $U^\dagger P U$ 的概率为 $tr(U^\dagger P U \rho) = tr(P U \rho U^\dagger)$. 而运行完 $\bar{q} := U \bar{q}$ 语句后, 程序状态正好是 $U \rho U^\dagger$, 此时满足 P 的概率为 $tr(P U \rho U^\dagger)$, 容易见到二者相等.

1.4 总结和展望

选择这个主题是因为我的研究方向正是程序语言 (PL) 方向中的形式化验证. 在某次课结束后我去食堂的路上想到了能否在量子计算中做形式化验证, 然后搜索了一些相关的论文. 很遗憾由于本人能力有限, 有一些论文截至本文完成仍未读完, 包括在分布式量子系统上做验证的工作^[10]以及将 Relational Logic 引入量子霍尔逻辑的工作^[11]. 最终呈现的论文看上去仅仅是在^[6]的基础上加了我的个人理解, 甚至出于篇幅和可理解性考虑略去了大量内容. 但我仍然很开心那天能想到这个 idea, 并且阅读这些有意思的 (虽然真的很难) 的论文, 并且在读论文的过程中学到了非常多有趣的知识!

第二章 量子物理学导论：这门课教会了我什么

首先我真的学到了非常多的量子力学知识性的东西：例如费米子和玻色子的区别，冷却原子的方式，薛定谔方程的形式，量子模拟和量子计算的前景等等等等。除此之外，我觉得很让我激动的还有一些理解上的改变。

比如量子叠加态的含义。我之前可能会认为叠加态无非只是在测量之前有概率为 0 也有概率为 1。这实际上其实还是传统概率式，或者说爱因斯坦式的理解方式。叠加态要比经典概率复杂得多。比如，在之前我可能并不知道怎么区分：一个以经典概率等概率取 0 或者 1 的 bit，和一个 $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ 存在的 qubit。我可能会认为如果不考虑坍缩效应的话它们的表现应该是完全一样的！但实则不然，回忆 Hadamard 门 $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ ，如果它施加到一个经典概率等概率取 0 或者 1 的 bit 上，或者说施加到一个混合态 $\frac{1}{2}|0\rangle\langle 0| + \frac{1}{2}|1\rangle\langle 1|$ 上，那我测量这个 bit 还是有 $\frac{1}{2}$ 概率为 1。但如果施加到一个叠加态 $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ 上，得到的结果其实是 $|0\rangle$ ，也就是有 100% 的概率会测到 0！这是我在这门课之前完全没有理解到的地方，意识到这点的时候我真的是超级激动。

姚老师上课讲授的量子力学历史使我对整个量子力学的发展动机和过程都有了非常清晰的认知。比如我以前一直有的一个错觉是德布罗意提出的物质波似乎并没有什么大不了的，感觉只是做了个无聊的形式扩展，但后来才发现这是有非常深刻道理的！例如当物质波波长并不显著小于原子间间距时，原子之间就会有波动性的影响。还有上课讲的比如当人们发现一个公式出现了一些无法处理的情况时，可能会考虑物态的转变，而我之前对物态的理解完全没深入到这一层。

贺老师的讲授则是让我原本对线性代数的理解转移到了物理上！比如“测量”实际上是一种线性算符，而由于位置算符和动量算符之间不可交换，意味着它们不可同步对角化，意味着测不准原理。感觉我之前对测不准原理的理解非常差，我甚至觉得只是技术上的问题使得“造不出那么小的尺子”，但算符视角真的使我茅塞顿开。另一件事就是，我之前并不理解为什么量子计算中的操作需要是 Unitary 算符，现在我明白这是因为薛定谔方程中的时间演化算符是 Unitary 的。以及我对 trace 在量子物理中代表的意义也有了非常深刻的理解。

总之非常好课！学会很多，感谢老师和助教和嘉宾们一学期的辛苦付出！感谢《量子物理学导论》这门课！

第三章 附录 A: 我目前对 trace 的理解

trace 实际上可以理解作为一种内积 (Frobenius 内积), 我们知道内积实际上可以看作向一个方向的投影. 在这种理解下, $tr(A) = tr(IA)$ 就可以理解为向整个空间的投影, 因此我们总会要求 density operator 满足 $trace = 1$, 以及 partial density operator 的 $trace$ 表示其能走到这里的概率. 而且 $tr(A\rho)$ 就可以理解为 ρ 到 A 上的投影, 这就能理解为什么说测量 A 结果确实是概率就是 $tr(A\rho)$.

现在还有一个问题是: 为什么 trace 可以理解作为一种内积呢? 这篇知乎回答^[12]给出了非常优秀的解释:

众所周知, 内积是一种双线性形式: $V \times V \rightarrow F$, 或者写作: $V \rightarrow V^*$. 也就是说: 一个线性空间到自身对偶空间的映射就可以引出一个内积. 现在我们来看方阵, 它们实际上是线性空间的自同态, 也就是 $\text{End } V$. 这种自同态的方式实际上也可以这么理解: 先选定一组基, 然后看当前向量对各个基的贡献. 也就是说它总将 $\vec{v} \mapsto \sum_i f_i(\vec{v})\vec{e}_i$, 这就给出 $\text{End } V \cong V \otimes V^*$. 然而此时注意到 $(\text{End } V)^* = V^* \otimes V \cong \text{End } V$, 这给出一个自然的 $\text{End } V \rightarrow (\text{End } V)^*$ 的映射, 这个映射引出的内积就是 trace.

参考文献

- [1] LEROY X, BLAZY S, KÄSTNER D, et al. CompCert - A Formally Verified Optimizing Compiler[C/OL]//ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress. Toulouse, France, 2016. <https://inria.hal.science/hal-01238879>.
- [2] ÖMER B. Classical Concepts in Quantum Programming[J/OL]. International Journal of Theoretical Physics, 2005, 44(7): 943-955. <http://dx.doi.org/10.1007/s10773-005-7071-x>. DOI: 10.1007/s10773-005-7071-x.
- [3] SANDERS J W, ZULIANI P. Quantum Programming[C]//BACKHOUSE R, OLIVEIRA J N. Mathematics of Program Construction. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000: 80-99.
- [4] SELINGER P. Towards a quantum programming language[J/OL]. Mathematical Structures in Computer Science, 2004, 14(4): 527-586. DOI: 10.1017/S0960129504004256.
- [5] ALTENKIRCH T, GRATTAJE J. A functional quantum programming language[C/OL]//20th Annual IEEE Symposium on Logic in Computer Science (LICS' 05). 2005: 249-258. DOI: 10.1109/LICS.2005.1.
- [6] YING M. Floyd-hoare logic for quantum programs[J/OL]. ACM Trans. Program. Lang. Syst., 2012, 33(6). <https://doi.org/10.1145/2049706.2049708>. DOI: 10.1145/2049706.2049708.
- [7] SVORE K, GELLER A, TROYER M, et al. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL[C/OL]//RWDSL2018: Proceedings of the Real World Domain Specific Languages Workshop 2018. ACM, 2018: 1-10. <http://dx.doi.org/10.1145/3183895.3183901>. DOI: 10.1145/3183895.3183901.
- [8] Cirq[EB/OL]. <https://quantumai.google/cirq>.
- [9] JAVADI-ABHARI A, TREINISH M, KRSULICH K, et al. Quantum computing with Qiskit [EB/OL]. 2024. <https://arxiv.org/abs/2405.08810>. arXiv: 2405.08810 [quant-ph].
- [10] FENG Y, LI S, YING M. Verification of Distributed Quantum Programs[J/OL]. ACM Trans. Comput. Logic, 2022, 23(3). <https://doi.org/10.1145/3517145>. DOI: 10.1145/3517145.
- [11] UNRUH D. Quantum relational Hoare logic[J/OL]. Proc. ACM Program. Lang., 2019, 3(POPL). <https://doi.org/10.1145/3290346>. DOI: 10.1145/3290346.
- [12] 上条当麻. 为什么会定义矩阵的迹? [EB/OL]. <https://www.zhihu.com/question/51293797/answer/2028616946434720324>.